PROJECT REPORT

DESIGN AND SIMULATION OF A CMOS XOR GATE USING LTspice

Implementation using four CMOS NAND gates

Submitted by
Aditya Ramakant Desai

Course: VLSI Design / Glow Logics Project
Simulation Tool: LTspice

Detailed academic report covering digital logic, CMOS transistor operation, NAND implementation, XOR derivation, transistor-level architecture, LTspice simulation, verification, applications and future scope.

DECLARATION

I declare that this report titled **Design and Simulation of a CMOS XOR Gate Using LTspice** has been prepared as part of my VLSI design learning. It explains the theoretical design and simulation procedure for a two-input XOR function implemented using four CMOS NAND gates. Any measured waveform, delay, power or voltage values in the final submission must be taken from the actual LTspice simulation.

Name: Aditya Ramakant Desai
Signature: _____
Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to the instructors and mentors associated with the VLSI learning programme for providing the opportunity to study digital circuit design and transistor-level implementation. I also acknowledge the developers of LTspice for providing a simulation environment that helps learners understand circuit behaviour before fabrication. I am thankful to my institution, classmates and family for their encouragement.

ABSTRACT

The Exclusive-OR (XOR) gate is a fundamental digital logic element used in arithmetic circuits, parity generation, error detection and comparison systems. It produces logic HIGH when its two inputs are different and logic LOW when they are equal. This project presents a two-input CMOS XOR gate implemented using four NAND gates. The internal signals are N1 = NAND(A,B), N2 = NAND(A,N1), N3 = NAND(B,N1), and Y = NAND(N2,N3).

Each two-input NAND gate is implemented at transistor level using two PMOS transistors in parallel in the pull-up network and two NMOS transistors in series in the pull-down network. Four NAND blocks therefore require sixteen MOSFETs in this conventional static CMOS implementation. The report discusses Boolean derivation, MOSFET operation, transistor connections, input pulse sources, transient analysis and truth-table verification in LTspice. The expected output is LOW for 00 and 11, and HIGH for 01 and 10. Actual simulation evidence must be added before submission.

TABLE OF CONTENTS

Chapter	Topic
1	Introduction and objectives
2	Digital logic fundamentals
3	CMOS technology and MOSFET operation
4	CMOS NAND gate design
5	XOR Boolean derivation
6	Transistor-level architecture
7	LTspice simulation procedure
8	Expected results and verification
9	Troubleshooting
10	Applications
11	Advantages, limitations and future scope

Chapter	Topic
12	Conclusion, references and appendices

CHAPTER 1: INTRODUCTION

1.1 Background

Digital electronics represents information using discrete logic levels, commonly called logic 0 and logic 1. Logic gates perform Boolean operations on these levels. AND, OR, NOT, NAND, NOR, XOR and XNOR gates form the basic building blocks of processors, microcontrollers, memories, communication systems and digital controllers.

At the physical level, logic gates are constructed from electronic devices. CMOS technology uses complementary NMOS and PMOS transistors. It is widely used in integrated circuits because of its low static power consumption in stable logic states, high input impedance and suitability for dense integration.

1.2 Project overview

This project designs a two-input XOR function using four NAND gates. NAND is a universal gate, which means that any Boolean function can be constructed using only NAND operations. The project connects Boolean algebra to transistor-level circuit implementation and uses LTspice to examine the circuit response.

1.3 Problem statement

The objective is to implement a two-input XOR logic function using CMOS NAND gates and study its behaviour through transient simulation. The circuit must produce HIGH when the inputs differ and LOW when the inputs are equal.

1.4 Objectives

- Understand XOR operation and its truth table.
- Derive XOR using only NAND gates.
- Construct CMOS NAND gates using PMOS and NMOS devices.
- Interconnect four NAND blocks to obtain XOR.
- Apply pulse inputs and perform transient analysis in LTspice.
- Compare stable output values with the expected truth table.
- Develop skills in schematic capture, simulation and debugging.

1.5 Scope

The work covers schematic-level design and transient simulation. It does not claim silicon fabrication, physical layout verification, post-layout extraction or laboratory measurement. These can be included as advanced extensions.

CHAPTER 2: DIGITAL LOGIC FUNDAMENTALS

2.1 Binary logic

Binary logic uses two states. Logic 0 is represented by a voltage near ground, and logic 1 by a voltage near the positive supply. Exact voltage ranges depend on the technology, supply and noise margins. In a simulation, input sources are configured to switch between selected LOW and HIGH voltage levels.

2.2 Basic logic operations

An AND gate is HIGH only when all inputs are HIGH. An OR gate is HIGH when at least one input is HIGH. A NOT gate inverts its input. NAND is the complement of AND, and XOR is HIGH when its two inputs differ.

A	B	AND	OR	NAND	XOR
0	0	0	0	1	0
0	1	0	1	1	1
1	0	0	1	1	1
1	1	1	1	0	0

2.3 XOR expression

The Boolean expression for a two-input XOR gate is:

```
Y = A■B + AB■
```

The first product term is true when A is LOW and B is HIGH. The second is true when A is HIGH and B is LOW. The output is therefore HIGH for unequal inputs.

2.4 XOR truth table

A	B	Y	Meaning
0	0	0	Inputs equal
0	1	1	Inputs different
1	0	1	Inputs different
1	1	0	Inputs equal

2.5 Universal gate

NAND is called a universal gate because NOT, AND, OR and more complex Boolean functions can be formed from NAND gates. This is useful in logic design when implementation is restricted to one gate family.

CHAPTER 3: CMOS TECHNOLOGY AND MOSFET OPERATION

3.1 CMOS structure

CMOS means Complementary Metal-Oxide-Semiconductor. A static CMOS logic gate contains a PMOS pull-up network and an NMOS pull-down network. The pull-up network connects the output to VDD when the output should be HIGH. The pull-down network connects the output to ground when the output should be LOW. The networks are designed to provide the required complementary switching behaviour.

3.2 NMOS transistor

An NMOS transistor forms a conductive channel when its gate-to-source voltage exceeds its threshold voltage. In digital logic, NMOS devices commonly form the pull-down network. A HIGH gate signal turns an NMOS device ON, allowing current to flow through the channel. A LOW gate signal ideally turns it OFF.

3.3 PMOS transistor

A PMOS transistor conducts when its gate voltage is sufficiently lower than its source voltage. PMOS devices are used in the pull-up network. A LOW gate input turns a PMOS device ON, providing a path from VDD toward the output. A HIGH gate input ideally turns it OFF.

3.4 Pull-up and pull-down networks

The pull-up network (PUN) consists of PMOS transistors. The pull-down network (PDN) consists of NMOS transistors. Series-connected devices require every device in the path to conduct. Parallel-connected devices require at least one branch to conduct. These series and parallel relationships determine the Boolean function.

3.5 Power consumption

In an ideal static CMOS gate, a stable logic state does not create a direct conducting path from VDD to ground, so static power is low. Real devices have leakage. During switching, output capacitance is charged or discharged and short-circuit current may briefly occur. A common approximation for dynamic power is $P_{dynamic} = \alpha C V_{DD}^2 f$, where α is switching activity, C is switched capacitance, VDD is supply voltage and f is switching frequency.

3.6 MOSFET terminals

A MOSFET has gate, drain, source and body/bulk terminals. For a conventional CMOS implementation, PMOS body is normally referenced to VDD and NMOS body to ground, subject to the selected model. LTspice MOSFET pin order and symbol orientation must be checked against the model. Do not infer electrical terminal identity solely from the visual top or bottom position of a symbol.

CHAPTER 4: CMOS NAND GATE DESIGN

4.1 NAND function

For two inputs A and B, the NAND output is the complement of their AND operation:

```
N = (A · B)■
```

The output is LOW only when both inputs are HIGH. For the other three input combinations, the output is HIGH.

4.2 CMOS NAND structure

A conventional two-input CMOS NAND uses two PMOS transistors in parallel in the pull-up network and two NMOS transistors in series in the pull-down network. The PMOS devices are parallel so that either LOW input can establish a pull-up path. The NMOS devices are series so that a complete pull-down path exists only when both inputs are HIGH.

4.3 NAND operation

A	B	PMOS behaviour	NMOS behaviour	Output
0	0	Both ON	Both OFF	1
0	1	At least one ON	One OFF	1
1	0	At least one ON	One OFF	1
1	1	Both OFF	Both ON	0

4.4 Why PMOS is parallel

A NAND output must remain HIGH if either input is LOW. A PMOS transistor turns ON when its gate is LOW. Parallel PMOS branches ensure that either input can activate a path from VDD to the output.

4.5 Why NMOS is series

The NAND output should become LOW only when both inputs are HIGH. An NMOS turns ON for a HIGH gate input. Series connection ensures that a complete path from output to ground exists only when both NMOS devices are ON.

4.6 Transistor count

Each two-input NAND uses two PMOS and two NMOS devices, giving four MOSFETs per NAND. Four NAND gates therefore use 4 × 4 = 16 MOSFETs, excluding any optional output buffer or test circuitry.

CHAPTER 5: XOR DESIGN AND BOOLEAN DERIVATION

5.1 Four-NAND implementation

The XOR function can be constructed using the following four NAND operations:

```
N1 = NAND(A,B) = (A · B)■

N2 = NAND(A,N1) = (A · N1)■

N3 = NAND(B,N1) = (B · N1)■

Y = NAND(N2,N3) = (N2 · N3)■
```

5.2 Step-by-step derivation

Start with N1 = (AB)■ = A■ + B■, using De Morgan’s theorem. The second output is N2 = (A N1)■. Substituting N1 gives N2 = (A(A■+B■))■. Since AA■ = 0, the expression inside the bar becomes AB■. Therefore N2 = (AB■)■ = A■ + B.

Similarly, N3 = (B N1)■ = (B(A■+B■))■. Since BB■ = 0, the expression inside the bar becomes A■B. Therefore N3 = (A■B)■ = A + B■.

The final output is Y = (N2N3)■. Substituting and simplifying gives Y = A■B + AB■, which is the XOR function.

5.3 Internal signal table

A	B	N1	N2	N3	Y
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	0

5.4 Signal flow

Inputs A and B feed NAND 1. Its output N1 is shared with NAND 2 and NAND 3. NAND 2 receives A and N1 and produces N2. NAND 3 receives B and N1 and produces N3. NAND 4 receives N2 and N3 and produces final output Y.

```
A,B → NAND 1 → N1
A,N1 → NAND 2 → N2
B,N1 → NAND 3 → N3
N2,N3 → NAND 4 → Y
```

CHAPTER 6: TRANSISTOR-LEVEL ARCHITECTURE

6.1 Overall structure

The design contains four two-input CMOS NAND blocks. Each block has a PMOS pull-up pair and an NMOS pull-down pair. The four blocks share the intended positive supply and ground reference, while their logic inputs and outputs are connected according to the NAND-only XOR architecture.

6.2 Device groups

NAND block	PMOS devices	NMOS devices	Output
NAND 1	M1, M2	M3, M4	N1
NAND 2	M5, M6	M7, M8	N2
NAND 3	M9, M10	M11, M12	N3
NAND 4	M13, M14	M15, M16	Y

6.3 Gate mapping

Block	First input gates	Second input gates
NAND 1	M1/M3 = A	M2/M4 = B
NAND 2	M5/M7 = A	M6/M8 = N1
NAND 3	M9/M11 = B	M10/M12 = N1
NAND 4	M13/M15 = N2	M14/M16 = N3

For each NAND block, the two PMOS devices are wired in parallel and the two NMOS devices in series. The gate mapping table identifies which logic signal drives each input pair. The output of each block is the common drain node of its pull-up and pull-down networks.

6.4 Supply and ground

- Connect the PMOS source terminals to the common VDD rail.
- Connect PMOS body terminals as required by the selected model, conventionally to VDD.
- Connect the bottom of each NMOS series stack to common GND.
- Connect NMOS body terminals as required by the model, conventionally to GND.
- Connect the negative terminals of both input sources to the same ground reference.

6.5 Intermediate net connections

N1 must connect to one input of NAND 2 and one input of NAND 3. It must not be shorted to N2, N3 or Y. N2 connects to one input of NAND 4, while N3 connects to the other input of NAND 4. The output of NAND 4 is Y. Clear net labels reduce accidental connections.

6.6 Design checks

A visual crossing of wires is not necessarily an electrical junction. Confirm junction dots or use net labels. Verify that no input is floating, all supplies are connected and the MOSFET body pins follow the model requirements. Inspect the generated netlist if the schematic appearance is ambiguous.

CHAPTER 7: LTSPICE SIMULATION METHODOLOGY

7.1 LTspice overview

LTspice is a SPICE-based circuit simulator used for analogue and mixed-signal analysis. It supports schematic capture, device models, transient analysis, DC operating-point analysis and waveform viewing. In this project, pulse voltage sources represent the digital inputs and transient analysis displays the output response over time.

7.2 Schematic preparation

- Create a new schematic in LTspice.
- Place the required PMOS and NMOS symbols for four NAND blocks.
- Wire each NAND with parallel PMOS devices and series NMOS devices.
- Connect the transistor gates using the mapping in Chapter 6.
- Label the intermediate nodes N1, N2 and N3 and final output Y.
- Add the VDD supply, ground and two independent input sources.
- Check model names and model directives before simulation.
- Save the schematic with a descriptive project filename.

7.3 Input pulse sources

The following pulse definitions are suggested for generating all four combinations of A and B:

```
A: PULSE(0 5 0 1n 1n 10m 20m)
B: PULSE(0 5 0 1n 1n 20m 40m)
Transient command: .tran 40m
```

Input A has a 20 ms period and 10 ms HIGH duration. Input B has a 40 ms period and 20 ms HIGH duration. Their combination cycles through all four input states during a complete interval, subject to the source phase and simulator timing. Both positive terminals must be separate nets A and B. Their negative terminals share ground.

7.4 Transient analysis

Transient analysis calculates node voltages as a function of time. The .tran 40m directive requests a simulation interval of 40 milliseconds. After the simulation, plot A, B and Y. During debugging, also plot N1, N2 and N3 to identify which stage first behaves incorrectly.

7.5 Simulation verification procedure

- Confirm the simulator completes without fatal errors.
- Confirm A and B switch between the intended voltage levels.
- Check intermediate nodes for floating or stuck values.
- Allow for propagation delay after each input transition.
- Compare stable output intervals with the XOR truth table.
- Save screenshots only after the actual simulation has been verified.

7.6 Project file organisation

Save the schematic as CMOS_XOR_PROJECT.asc. Save a schematic screenshot and a separate waveform screenshot. If custom model files are required, include them in the project folder and explain their purpose in the README. Keep filenames clear so another person can reproduce the simulation.

CHAPTER 8: EXPECTED RESULTS AND VERIFICATION

8.1 Expected output

The expected output is LOW for input combinations 00 and 11, and HIGH for 01 and 10. This behaviour is the defining property of XOR.

A	B	Expected Y	Interpretation
0	0	0	Inputs equal
0	1	1	Inputs differ
1	0	1	Inputs differ
1	1	0	Inputs equal

8.2 Waveform interpretation

When both inputs are LOW, the output should be LOW. When B becomes HIGH while A is LOW, the output should become HIGH after the NAND stages respond. When A becomes HIGH while B is HIGH, the output should return LOW. When B returns LOW while A is HIGH, the output should become HIGH. When both inputs are LOW again, the output should return LOW.

Small transition delays and short glitches can occur because the four cascaded NAND stages do not switch instantaneously. These effects depend on transistor models, device dimensions, load capacitance, supply voltage and simulation settings. Stable logic levels are the primary criterion for this introductory verification.

8.3 Observed results table

Complete the observed column using the actual waveform. Do not present expected values as measured results.

A	B	Expected Y	Observed Y	Pass/Fail
0	0	0	_____	_____
0	1	1	_____	_____
1	0	1	_____	_____
1	1	0	_____	_____

8.4 Performance measurements

A more advanced evaluation may measure propagation delay, rise time, fall time, average supply current and average power. These values must be extracted from the actual simulation. For propagation delay, use consistent input and output threshold crossings, commonly 50% of the logic swing. Low-to-high and high-to-low delays may differ because pull-up and pull-down strengths are not necessarily equal.

Parameter	Description	Measured value
VDD	Supply voltage	_____
VOH	Output HIGH level	_____
VOL	Output LOW level	_____
tpLH	Low-to-high propagation delay	_____
tpHL	High-to-low propagation delay	_____
tr	Output rise time	_____
tf	Output fall time	_____

Parameter	Description	Measured value
Pavg	Average power	_____

CHAPTER 9: TROUBLESHOOTING AND DESIGN CONSIDERATIONS

9.1 Floating nodes

A floating node does not have a defined voltage under the simulator's circuit rules. It may cause warnings or unexpected output. Check that each transistor gate is driven by an intended logic signal and that supply, ground and output connections are complete.

9.2 Incorrect NAND topology

Connecting NMOS devices in parallel or PMOS devices in series changes the intended NAND function. The correct two-input CMOS NAND structure uses parallel PMOS devices and series NMOS devices. Test each NAND block separately before integrating all four blocks.

9.3 Incorrect intermediate connections

The critical connections are N1 to NAND 2 and NAND 3, N2 to NAND 4, and N3 to NAND 4. Accidentally joining intermediate nets can destroy the XOR logic. Use unique net labels and inspect the netlist when necessary.

9.4 Supply and body connections

A missing supply or ground connection can leave an output undefined. Incorrect body connections may also affect operation depending on the device model. Check the selected MOSFET symbol and model documentation.

9.5 Input sources in parallel

If both input voltage sources are connected to the same positive net, LTspice may report that the sources are in parallel. The positive terminal of source A must connect to net A, and the positive terminal of source B must connect to net B. Their negative terminals may share ground.

9.6 Model conflicts

A model name should be defined consistently. Duplicate or conflicting model definitions can cause warnings or unexpected device behaviour. Review schematic directives and included libraries. Use unambiguous model names compatible with the selected symbols.

9.7 Output stuck HIGH or LOW

A constant output can indicate incorrect transistor pin mapping, missing supply, floating inputs, a wrong model or an incorrect connection. Debug one NAND block at a time and confirm its truth table before reconnecting the full XOR circuit.

9.8 Recommended debugging practice

- Use clear net names and avoid ambiguous wire crossings.
- Test each NAND block independently.
- Inspect the netlist when visual wiring is uncertain.
- Plot intermediate signals to locate the first failing stage.
- Change one issue at a time and rerun the simulation.
- Keep a known-good copy before major edits.

CHAPTER 10: APPLICATIONS OF XOR GATES

10.1 Arithmetic circuits

XOR gates are used in half adders and full adders. In a half adder, the sum is A XOR B and the carry is A AND B. In a full adder, XOR operations combine the two input bits and carry-in to generate the sum bit.

10.2 Parity generation and checking

XOR gates generate and check parity bits for simple error detection. A parity generator combines data bits using XOR. A receiver can recompute parity and compare it with the received parity bit. Parity detects some errors but does not identify or correct every possible error.

10.3 Comparators

XOR can indicate whether two individual bits differ. LOW means equal and HIGH means different. Multiple XOR gates can be combined to compare multi-bit values.

10.4 Digital communication

XOR is used in scrambling, encoding and related digital communication operations. A data bit can be combined with a control or sequence bit. Applying the same XOR sequence again can recover the original bit.

10.5 Control and processing systems

XOR logic is found in arithmetic units, control circuits, checksum logic and programmable digital systems. Transistor-level study helps explain how these high-level functions are realised in integrated circuits.

CHAPTER 11: ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

11.1 Advantages

- Demonstrates a complete logic function using a universal gate.
- Connects Boolean algebra with transistor-level CMOS design.
- Uses repeatable NAND building blocks.
- Can be verified with four input combinations.
- Provides practical LTspice schematic and waveform experience.

11.2 Limitations

- Four cascaded NAND stages use more transistors than some dedicated XOR structures.
- Cascaded stages introduce propagation delay.
- The circuit is sensitive to incorrect wiring and model selection.
- Schematic simulation does not include every physical layout effect.
- Performance figures cannot be claimed without extracting them from simulation.

11.3 Future scope

The design can be extended by comparing the four-NAND XOR with a dedicated CMOS XOR topology. Different transistor sizes can be evaluated to study area, delay and power trade-offs. A capacitive load can be added to examine output drive capability. The same function can also be described in Verilog and synthesised to compare logic-level and transistor-level implementations.

An advanced VLSI extension may include layout design, design-rule checking, layout-versus-schematic verification and parasitic extraction using a suitable process design kit. Post-layout simulation can then be compared with schematic-level results.

11.4 Learning outcomes

The project develops understanding of Boolean simplification, universal gates, CMOS pull-up and pull-down networks, MOSFET pin identification, schematic wiring, transient analysis and waveform interpretation. These are foundational skills for further study in digital VLSI design.

CHAPTER 12: CONCLUSION

This project report presents the design methodology for a two-input CMOS XOR gate using four NAND gates. The Boolean function is derived using NAND operations and implemented using sixteen MOSFETs arranged as four static CMOS NAND blocks. Each NAND uses parallel PMOS devices in the pull-up network and series NMOS devices in the pull-down network.

The report explains the digital logic, MOSFET operation, signal interconnection and LTspice simulation procedure. The expected truth table confirms that the output is LOW for equal inputs and HIGH for different inputs. Successful completion must be demonstrated using the actual LTspice waveform and by checking all four stable input combinations. The design provides a foundation for further work in CMOS logic, arithmetic circuits and power-delay analysis.

REFERENCES

- Neil H. E. Weste and David Money Harris, CMOS VLSI Design: A Circuits and Systems Perspective, Addison-Wesley.
- Jan M. Rabaey, Anantha Chandrakasan and Borivoje Nikolic, Digital Integrated Circuits: A Design Perspective, Prentice Hall.
- M. Morris Mano and Michael D. Ciletti, Digital Design, Pearson.
- LTspice documentation and help resources, Analog Devices.
- Course notes and learning material provided as part of the VLSI design programme.

APPENDIX A: QUICK DESIGN SUMMARY

Item	Design detail
Function	Two-input XOR
Implementation	Four CMOS NAND gates
NAND topology	Two PMOS in parallel; two NMOS in series
MOSFET count	16
Internal nets	N1, N2, N3
Final output	Y = A XOR B
Input A	PULSE(0 5 0 1n 1n 10m 20m)
Input B	PULSE(0 5 0 1n 1n 20m 40m)
Transient command	.tran 40m

APPENDIX B: TRANSISTOR CONNECTION CHECKLIST

- NAND 1: M1/M2 PMOS parallel; M3/M4 NMOS series; gates A and B; output N1.
- NAND 2: M5/M6 PMOS parallel; M7/M8 NMOS series; gates A and N1; output N2.
- NAND 3: M9/M10 PMOS parallel; M11/M12 NMOS series; gates B and N1; output N3.
- NAND 4: M13/M14 PMOS parallel; M15/M16 NMOS series; gates N2 and N3; output Y.
- All four blocks use the intended common VDD and GND references.
- Input sources have separate positive nets A and B and common negative ground.
- No input or intermediate logic node is left floating.

APPENDIX C: SUBMISSION CHECKLIST

- Final LTspice schematic (.asc).
- Schematic screenshot showing all four NAND blocks and labels.
- Waveform screenshot showing A, B and Y.
- Observed-results table completed from the actual waveform.
- This PDF report.
- README file describing the project, tools, truth table and files.
- Any custom model/library files required to reproduce the simulation.
- Public GitHub repository link.

Important submission note: Insert the actual LTspice schematic and waveform screenshots into the report or attach them as separate repository files. Complete the observed-results and performance tables only with values obtained from your own simulation.